

C. elegans Longevity Simulator

Sprint 0 Report — Scope, Technology Stack, and Prototype

Team members: Michael LeBlanc • Tyler Bullard • Caleb Walton

GitHub repository: github.com/mxl6w94/C.elegans-longevity-simulation

Date: October 5, 2026

1. GitHub Repository

Repository: <https://github.com/mxl6w94/C.elegans-longevity-simulation>

Project primer (plain-language walkthrough, hosted on GitHub Pages): mxl6w94.github.io/C.elegans-longevity-simulation/PRIMER.html

2. Feature List — Preliminary Scope

The project is a web tool for researchers and students studying *C. elegans* longevity. A user selects genes from a curated list and sees a predicted lifespan, growth rate, and a visual of the worm at a chosen age. Predictions are produced by a transcriptomics-constrained flux balance analysis (FBA) pipeline built on the iCEL1314 metabolic model and the GSE52340 microarray dataset. The preliminary scope totals 18 features: 8 critical, 6 important, 4 useful.

1. **(Critical)** The user opens the web application in a browser.
2. **(Critical)** The user selects one or more genes from the curated gene list, or selects the wild-type baseline (no mutations) as its own option.
3. **(Critical)** The user submits the gene selection to request a prediction.
4. **(Critical)** The system displays the predicted life expectancy for the selected genes.
5. **(Critical)** The system displays the predicted growth rate for the selected genes.
6. **(Critical)** The user views the predicted survival curve as a graph.
7. **(Critical)** The user selects an age within the predicted lifespan and views an image of the worm at that age.
8. **(Critical)** The system displays an error message when input is invalid: no gene selected, an unsupported gene combination, an age outside the lifespan range, or the server being unreachable.
9. **(Important)** The user toggles a selected gene on or off and sees the prediction update against the wild-type baseline.
10. **(Important)** The user reads a short description of what each gene in the curated list does.
11. **(Important)** The user downloads the prediction results (survival curve, growth rate, flux table) as CSV files.
12. **(Important)** The user saves a prediction and retrieves it later.
13. **(Important)** The system reports when the model finds no feasible solution for the selected genes.
14. **(Important)** The researcher compares predicted lifespans against observed historical lifespan data to measure accuracy.
15. **(Useful)** The user searches the gene list by name or identifier.

16. **(Useful)** The user changes the selected age and the worm image updates in place.
17. **(Useful)** The system displays a predicted body size or weight for the worm.
18. **(Useful)** The user shares a saved prediction with others through a link.

3. Use Case Diagram

The diagram below covers the preliminary-scope features using standard UML use-case notation, with two actors (**Biologist**, the end user, and **Admin**, who manages genes and users), `<<include>>` relationships for mandatory sub-steps (e.g. every prediction-related action includes logging in), and `<<extend>>` relationships for optional/exceptional paths (e.g. an invalid gene request or an age outside the lifespan range).

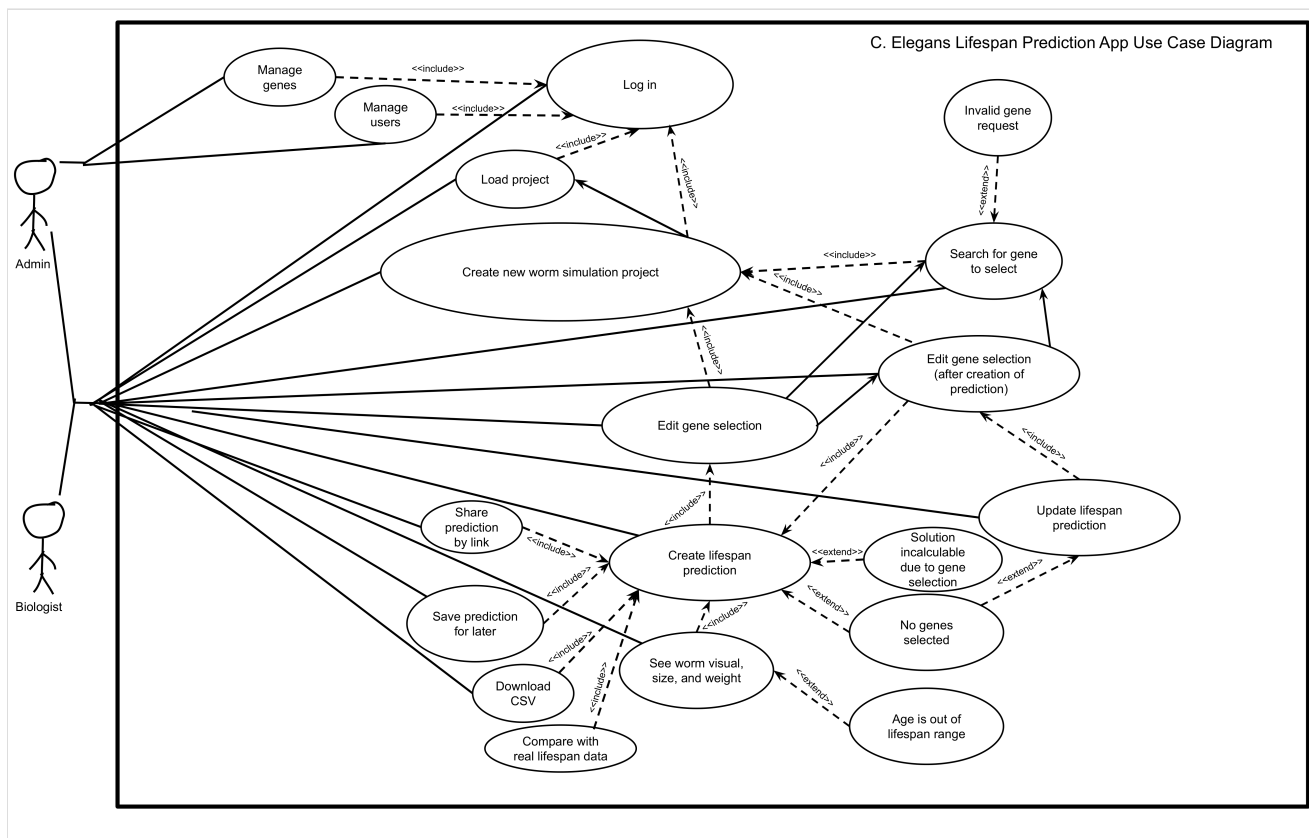


Figure 1. C. elegans Lifespan Prediction App — use case diagram.

4. Development Environment Report

This section evaluates alternatives for each major component of the stack and justifies the final selection. All scoring is on a 1 (poor) to 5 (excellent) scale per criterion; the weighted total is $\sum (\text{score}/5 \times \text{weight})$, out of 100.

4.1 Evaluation Criteria

Criterion	Weight	What it means for this project
Integration	25%	Works directly with the Python pipeline, COBRApy, and the CSV results, with no translation layer.
Team experience / learning curve	20%	The team already knows it, or can learn it within a few weeks.
Delivery time	15%	Gets critical features 1–8 working inside the semester.
Community support	15%	Documentation, tutorials, maintained packages, help-forum answers.
Scalability	10%	Handles more genes, more users, and a more complex interface later.
Security	10%	Input validation, escaping, protection against common web attacks, safe storage.
Cost	5%	Free to develop with and cheap or free to host.

4.2 Comparison of Alternatives

Frontend

Criterion (weight)	HTML/CSS/JS + Jinja + Chart.js	Vue 3	React
Integration (25%)	5	4	3
Team experience (20%)	5	3	3
Delivery time (15%)	5	4	3
Community support (15%)	4	4	5
Scalability (10%)	3	4	5
Security (10%)	4	4	4
Cost (5%)	5	5	5
Weighted total	91	77	74

Choice: HTML, CSS, and JavaScript (ES6+) rendered through Jinja templates, with Chart.js for the survival curve. No JavaScript framework. Flask serves the templates directly, and a small amount of JavaScript calls the REST API so results update without a full page reload. Jinja auto-escapes output, which covers the most common injection risk. React scores higher on scalability and community support and remains the fallback if the interface grows into something like a side-by-side comparison dashboard — the REST API in 4.2 (Backend) keeps that migration open without a backend rewrite. Given the one-semester timeline and the team's existing HTML/CSS/JS experience, the no-framework approach was chosen.

Backend and API

Criterion (weight)	Python + Flask	Python + FastAPI	Python + Django
Integration (25%)	5	5	4
Team experience (20%)	5	4	2
Delivery time (15%)	5	4	3
Community support (15%)	5	4	5
Scalability (10%)	3	4	4
Security (10%)	3	4	5
Cost (5%)	5	5	5
Weighted total	92	86	75

Choice: Python with Flask, exposing a REST API that returns JSON. The backend must be Python because the prediction engine is COBRAPy, a Python-only library. Flask wins on integration and delivery time — a working prototype already imports the `fluxworm` package directly (see Section 7.3). FastAPI is the closest alternative, with built-in request validation and automatic API docs, and remains an option if validation becomes a burden; its async support matters less here since FBA computation is CPU-bound, not I/O-bound. Django's admin site and ORM are more than this project needs.

API approach: REST over HTTP with JSON bodies was chosen over GraphQL. The planned surface is six fixed, well-understood endpoints (gene list, run/fetch a prediction, survival-curve points, worm image by age, CSV export) that don't need GraphQL's flexible querying, and REST is simpler for the team to test.

Database

Criterion (weight)	SQLite (relational)	PostgreSQL (relational)	Flat CSV/JSON files (current)
Integration (25%)	5	4	5
Team experience (20%)	4	4	5
Delivery time (15%)	5	3	4
Community support (15%)	5	5	3

Scalability (10%)	3	5	1
Security (10%)	3	5	2
Cost (5%)	5	4	5
Weighted total	88	84	77

Choice: a relational database, SQLite, accessed through SQLAlchemy. The data — genes, genotypes, predictions, survival-curve points, saved runs — has a fixed, linked-by-ID shape that fits relational tables better than a document store, which rules out something like MongoDB. SQLite ships with Python and needs no server, which keeps setup identical across every team machine. SQLAlchemy makes a later move to PostgreSQL mostly a connection-string change, which is planned if the app is hosted for many concurrent users. Flat files (the pipeline's current output format) work for precomputed results but can't safely handle saved predictions or concurrent writers.

Additional Tools

Area	Choice	Alternatives considered	Why
Unit and API tests	pytest + Flask test client	unittest	Already used for the pipeline's 19 unit tests; the Flask test client exercises the API without starting a server.
Acceptance tests	pytest-bdd	behave	User stories are already written as Given/When/Then, so each scenario converts to a test almost word for word.
End-to-end browser tests	Playwright (Python)	Selenium, Cypress	Tests the real pages in CI; Cypress would require a separate JavaScript test stack.
CI/CD	GitHub Actions	GitLab CI, Jenkins	Free for this repository, configured in-repo as YAML, runs on every push and pull request; Jenkins needs a server to maintain.
Hosting	AWS EC2	PythonAnywhere, a university VM	Low-cost Python hosting that deploys from GitHub; Flask's built-in server is development-only.
Containers (optional)	Docker	None	Pins the solver and Python versions so the hosted app matches every dev machine.
Documentation	Markdown in-repo, GitHub Pages	None	The project primer is already published on GitHub Pages.

4.3 Final Technology Stack and Justification

Component	Final choice
-----------	--------------

Frontend language	HTML5, CSS3, JavaScript (ES6+)
Frontend framework	None — Jinja templates, plus Chart.js for graphs
Backend language	Python 3.11
Backend framework	Flask, rendering templates and calling the pipeline directly
API	REST, JSON over HTTP
Database	Relational — SQLite through SQLAlchemy, with PostgreSQL as the scale-up path
Modeling engine	fluxworm pipeline (COBRApy + GLPK solver), results precomputed per genotype
Testing	pytest, Flask test client, pytest-bdd, Playwright
CI/CD	GitHub Actions, deploying to an AWS EC2 instance
Supporting tools	Git/GitHub, Docker (optional), GitHub Pages

The final stack keeps everything in Python except the browser code, which matches the team's existing experience, the already-working pipeline, and a one-semester timeline. It also removes the biggest delivery risk for a project like this: translating prediction logic between languages. Keeping the REST API and SQLAlchemy ORM in place leaves React or PostgreSQL open as later upgrades without a rewrite of the backend logic.

Proposed development environment: Windows 11 (team machines; macOS/Linux supported via `Launcher.sh`), VS Code with Python/Pylance/Jinja extensions, Python 3.11+ in a per-project virtual environment created by `Launcher.py`, local server via `flask --app app run --debug` at `http://127.0.0.1:5000`, Chrome/Firefox dev tools, Git with one feature branch per change and peer-reviewed pull requests into `main`.

4.4 References

1. Evaluation and Selection of Off-the-Shelf Software (course reference document) — used as the basis for the weighted-criteria evaluation method applied in Section 4.2.
2. Flask documentation. Pallets Projects. <https://flask.palletsprojects.com/>
3. COBRApy documentation. <https://cobrapy.readthedocs.io/>
4. SQLAlchemy documentation. <https://docs.sqlalchemy.org/>
5. Chart.js documentation. <https://www.chartjs.org/docs/>
6. GitHub Actions documentation. <https://docs.github.com/actions>
7. Yilmaz, L. S., & Walhout, A. J. M. (2016). Metabolic network modeling with model organism genome-scale reconstructions: iCEL1314. *Worm research community reference for the iCEL1314 model used as the project's metabolic network*.
8. Chen, D. et al. (2013). Germline signaling mediates the synergistically prolonged longevity produced by double mutations in *daf-2* and *rsks-1* in *C. elegans*. *Cell Reports* — source of the biological phenotype (GSE52340 dataset) the pipeline models.

9. AI-tool disclosure: Google Gemini was used to help refine requirements-interview questions (Activity 2); OpenAI ChatGPT was used to generate comparison user stories and acceptance criteria for review against the team's own versions (Activity 3). All AI-assisted content was reviewed, compared against team-authored material, and revised by the team before inclusion.

5. User Stories and Acceptance Criteria

The team selected three critical user stories for prototyping, one per member. Each was revised after comparison against an AI-generated version (see `Activity_3_UserStoriesandAcceptanceCriteria.md` for the full side-by-side comparisons); the revisions below reflect the team's final, agreed wording and are the basis for the story-point estimates in Section 6 and the prototype work in Section 7.

Story 1 — Web-accessible UI for gene selection and predictions *(Tyler Bullard)*

As a user, I want to access the app's web page, select genes, and view a lifespan/growth-rate prediction, **so that** I can run and read a prediction without any setup beyond a browser.

Access — Success. Given the user has a browser open, when the user enters the web address, then the website page is displayed.

Access — Failure. Given the user has a browser open, when the user enters the web address and the server is down or unreachable, then an error message is displayed indicating the service is temporarily unavailable.

Gene selection — Success. Given the user is on the webpage, when the user selects a set of genes, then a prediction is made.

Gene selection — Failure (no genes selected). Given the user is on the webpage, when the user submits a request without selecting any genes, then no prediction is made and the user is prompted to select at least one gene.

View prediction — Success. Given a prediction has been made, when the user requests to view it, then the prediction data is shown on the interface.

View prediction — Failure. Given no prediction has been made, when the user requests to view the prediction, then a message indicates no prediction exists and prompts gene selection.

Revision adopted from AI comparison: gene search by name/identifier (feature 15) was accepted into the backlog as a follow-on scenario, with duplicate-selection and search-timeout failure paths, since it strengthens the same feature area without changing scope.

Story 2 — Gene toggles with growth-rate response *(Michael LeBlanc)*

As a user, I want the model to accurately predict a *C. elegans* growth rate based on chosen genes, and to toggle genes on/off and compare the result against the wild-type baseline, **so that** I can see how each gene affects the prediction.

Genes chosen — Success. Given the user is on the webpage, when the user selects one or more genes to simulate, then the predicted growth rate and life expectancy are displayed.

Genes chosen — Failure (invalid gene identifier). Given the user is on the webpage, when the user enters a gene identifier that does not map to the underlying metabolic reconstruction, then no prediction is generated and an error prompts the user to verify the gene name.

Genes chosen — Failure (model infeasibility). Given the user is on the webpage, when the selected gene combination results in an unsolvable or biologically non-viable state, then a message indicates the simulation failed to find a valid solution.

Toggle — Success. Given a baseline prediction is loaded, when the user toggles a selected gene off, then the model recalculates and displays the updated growth rate.

Toggle — Failure (no metabolic impact). Given a baseline prediction is loaded, when the user toggles a gene that constrains no active reactions under current conditions, then the displayed growth rate is unchanged and a tooltip explains the knockout has no effect.

Revision adopted from AI comparison: the team agreed to integrate the AI-suggested criterion that growth-rate constraints should be driven by transcriptomic differential-expression data rather than binary knockouts alone — this is how the actual `fluxworm` pipeline (Section 7.2) already works, so the acceptance criteria were revised to match the real constraint-building logic instead of a simplified on/off model.

Story 3 — Worm visual at a selected biological age (*Caleb Walton*)

As a user in the field of biology, I want to see an image of what a worm with a specific age and gene set would look like, so that I have a visual of the worm at that biological age relative to chronological time.

Image generation — Success. Given the user has selected genes and generated a predicted lifespan, and is on the prediction graph with the option to view the worm, when the user selects an age within the predicted lifespan range and confirms, then an image of the worm's predicted appearance is displayed.

Age out of range — Failure. Given the same starting state, when the user selects an age not within the predicted lifespan range and confirms, then an error is displayed signaling the age is out of range.

Live update — Success. Given the user is viewing the visual representation, when the user changes the selected biological age, then the display updates to the worm's predicted appearance at the new age.

Revision adopted from AI comparison: the AI version's single combined scenario was split into the two independent scenarios above (initial generation vs. in-place update) for clarity, and the out-of-range failure scenario was kept as its own case rather than folded into the success path, matching the team's original structure.

6. Estimated Story Points

Estimates use a Fibonacci-like scale (1, 2, 3, 5, 8, 13) sized relative to the simplest critical feature, "open the web app" (1 point). Estimates reflect effort *and* uncertainty, not just lines of code — a story with a well-understood UI but an uncertain backend dependency scores higher than its UI alone would suggest.

User story	Points	Rationale
Story 1 — Web-accessible UI for gene selection and predictions	5	Mostly well-understood Flask/Jinja routing and form handling; a minimal version already exists (Section 7.3). Points above the minimum account for input validation, the "no genes selected" and "server unreachable" error paths, and the gene-search follow-on scenario.
Story 2 — Gene toggles with growth-rate response	8	Backend-heavy: each toggle has to re-run the transcriptomics-constrained FBA pipeline (or look up a precomputed genotype) and surface model infeasibility and no-impact cases correctly. The modeling logic exists in <code>core/</code> , but wiring live toggles to it, instead of a single static form submit, is new integration work with real uncertainty around solver behavior on edge-case gene combinations.
Story 3 — Worm visual at a selected biological age	8	No visual assets or age-interpolation logic exist yet. Requires producing or selecting images per genotype/age bucket, range validation against the genotype's predicted lifespan, and an in-place update on age change — the most novel, least-prototyped of the three stories.
Total (3 stories)	21	

7. Prototype Demonstration

7.1 User Interface Design (Low-Fidelity Wireframe)

The wireframe below lays out the target interface for all 18 preliminary-scope features (numbered callouts match the feature list in Section 2): a gene-selection panel, a prediction panel with the survival-curve graph, and a worm-at-age viewer.

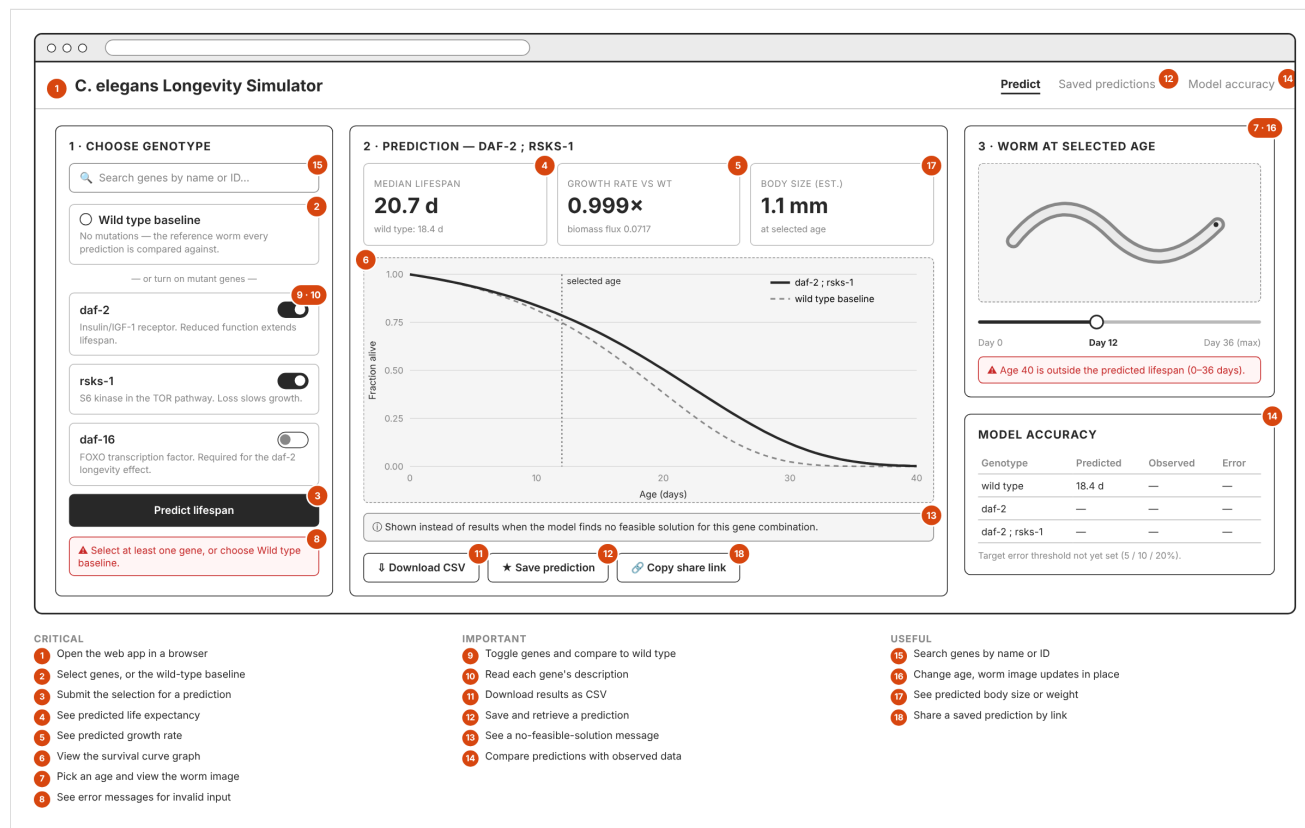


Figure 2. Low-fidelity wireframe — prediction page, with feature callouts.

7.2 Initial Architectural Design

The architecture keeps the prediction logic in Python end to end. The browser talks to a thin Flask layer, which calls the `fluxworm` pipeline directly rather than through a network hop; the pipeline's precomputed, per-genotype results are read from CSV today, with SQLite/SQLAlchemy planned for saved and shared predictions (Section 4.3).

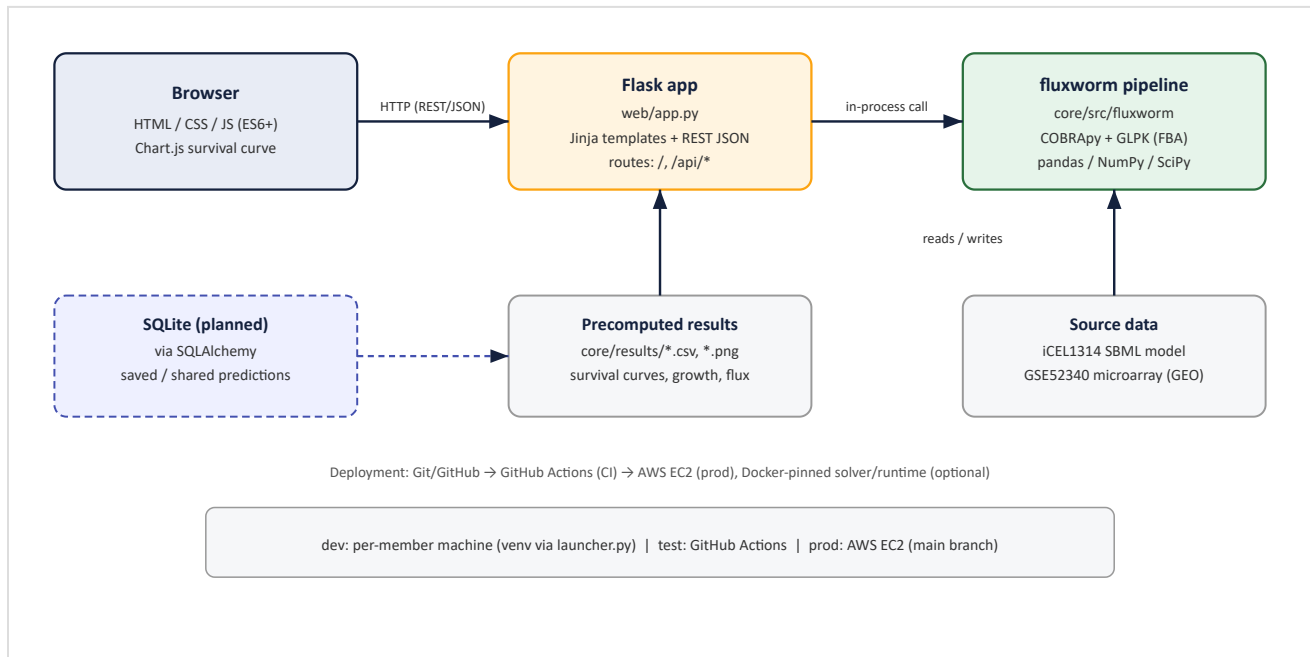


Figure 3. Initial architectural design — request flow, data sources, and environments.

7.3 Screenshots from the Implemented User Stories

To familiarize the team with the chosen environment, the Flask prototype in `web/app.py` was run end to end: it loads the pipeline's precomputed `survival_curves_illustrative.csv` and `growth_comparison.csv`, serves the gene-selection form from Story 1, computes a median lifespan and growth-rate figure for the selected genotype (Story 2), and returns a validation error for an unsupported combination. The screenshots below are captured directly from the running prototype, not mockups.

C. elegans lifespan predictor

☐ daf-2 mutant
 ☐ rsk-1 mutant
 ☐ daf-16 mutant

Predict

Figure 4. Prototype — initial page, gene-selection form (Story 1, feature 1 & 2).

C. elegans lifespan predictor

☐ daf-2 mutant
☐ rsk-1 mutant
☐ daf-16 mutant

Predict

daf-2

Median lifespan (illustrative units): 20.85

Growth vs. wild type: 0.9988

Figure 5. Prototype — successful prediction for the `daf-2` genotype: median lifespan and growth rate vs. wild type (Story 2, features 3–5).

C. elegans lifespan predictor

☐ daf-2 mutant
☐ rsk-1 mutant
☐ daf-16 mutant

Predict

That gene combination isn't in the dataset.

Figure 6. Prototype — validation error for a gene combination outside the dataset (Story 1 & 2, feature 8).

The worm-at-age visual (Story 3) is not yet implemented in the running prototype; Section 7.1's wireframe and Section 6's story-point estimate (8 points, the highest of the three) reflect that it is the least mature of the three stories going into Sprint 1.

8. Team Contribution List

Member	Contributions
Michael LeBlanc	Built the <code>fluxworm</code> transcriptomics-constrained FBA pipeline (<code>core/</code> : model loading, microarray parsing, constraint building, FBA runner, 19 unit tests); ran the requirements-elicitation interviews (Activity 2); authored the gene-toggle/growth-rate user story and acceptance criteria (Story 2, Activity 3); README updates; assembled this Sprint 0 report.
Tyler Bullard	Built and tested the Flask web prototype (<code>web/app.py</code> , templates); authored the development-environment evaluation and final stack justification (Section 4, <code>PreliminaryDevEnvironment.md</code>);

	designed the low-fidelity wireframe (Figure 2); authored the web-UI user story and acceptance criteria (Story 1, Activity 3); README feature-list formatting.
Caleb Walton	Constructed and revised the use case diagram (Figure 1, Section 3); authored the worm-visual user story and acceptance criteria (Story 3, Activity 3), including the AI-comparison revision.